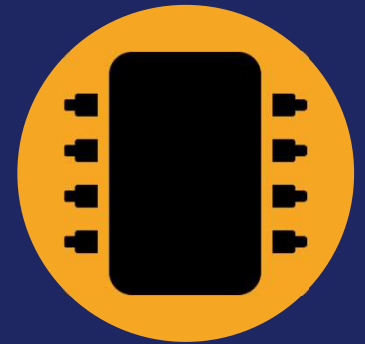


SISTEMAS OPERACIONAIS

Gerenciamento de Memória

Uma síntese do Capítulo 3 de "Sistemas Operacionais Modernos", de Andrew S. Tanenbaum e Herbert Bos: das abstrações de memória à memória virtual, paginação e segmentação.



Agenda



01 As duas visões do SO aplicadas à memória

02 Sistemas sem abstração de memória

03 Espaços de endereçamento e swapping

04 Memória virtual e paginação

05 Algoritmos de substituição de páginas

06 Thrashing e o modelo do conjunto de trabalho

07 Questões de projeto e de implementação

08 Segmentação e síntese final

O Sistema Operacional e a Memória



Máquina Estendida

O SO oculta as limitações físicas da RAM e oferece a cada processo a abstração de um espaço de endereçamento próprio, como se a memória fosse grande, rápida e privada.



Gerenciador de Recursos

Cabe ao SO decidir que partes da memória atribuir a cada processo, quando trocar dados com o disco e como arbitrar a disputa por um recurso físico escasso.

Hierarquia de memória: memória pequena, rápida e cara (cache, RAM) combinada com memória grande, lenta e barata (disco) — gerenciada automaticamente pelo SO por meio da memória virtual.

Sistemas sem Abstração de Memória



Nos primeiros sistemas — e em sistemas embarcados simples — o programa enxerga diretamente a memória física, com endereços absolutos. Apenas um processo pode ser executado por vez.

Overlays: sem memória virtual, o próprio programador dividia o programa em módulos que se revezavam na memória — solução manual e trabalhosa, superada pela memória virtual.

SO na RAM baixa

Mainframes e minicomputadores antigos

SO na ROM (parte alta)

Sistemas embarcados

Drivers na ROM alta + SO na RAM baixa

Alguns PCs antigos

Espaços de Endereçamento



- Conjunto de endereços que um processo pode usar, independente da posição física real na memória.
- Resolve dois problemas ao mesmo tempo: proteção (um processo não acessa a memória de outro) e realocação (o programa roda em qualquer posição física).
- Implementação clássica: registradores de base e limite — cada acesso soma o endereço à base e verifica o limite em hardware.
- Custo: uma soma e uma comparação a cada acesso à memória, exigindo suporte de hardware dedicado.

Endereço Virtual (gerado pelo processo)

+ registrador base

Endereço Físico

verificado contra registrador limite

Acesso permitido à RAM

Swapping e Gerenciamento do Espaço Livre



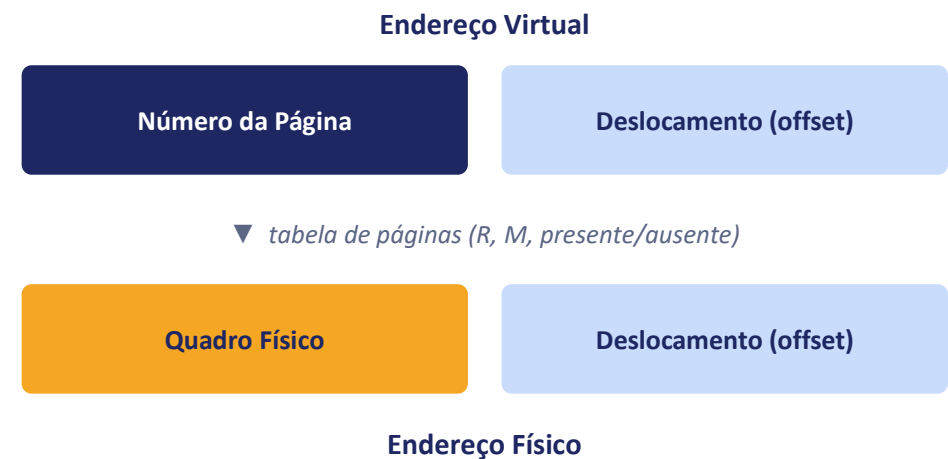
Quando a memória exigida pelos processos excede a RAM física, o SO pode mover processos inteiros entre memória principal e disco (swapping). O espaço livre é controlado por mapas de bits ou listas encadeadas.

Algoritmo	Estratégia de busca
First fit	Usa o primeiro buraco grande o suficiente
Next fit	Como o first fit, mas retoma a busca de onde parou
Best fit	Procura o menor buraco suficiente (mais lento; gera fragmentos inúteis)
Worst fit	Usa o maior buraco disponível (desempenho ruim na prática)
Quick fit	Listas separadas por tamanhos comuns (busca rápida; fusão mais complexa)

Memória Virtual e Paginação



- O espaço de endereçamento virtual é dividido em páginas de tamanho fixo; a memória física, em molduras (page frames) do mesmo tamanho.
- A MMU traduz endereços virtuais em físicos usando a tabela de páginas.
- Cada entrada guarda: número do quadro físico, bit presente/ausente, bits de proteção, bit de referência (R) e bit de modificação (M).
- Falta de página (page fault): a página referenciada não está na memória — o SO escolhe uma vítima e busca a página no disco.



TLB e Tabelas de Páginas Grandes



TLB (Translation Lookaside Buffer): pequeno cache dentro da MMU com as traduções mais recentes, evitando acessar a tabela de páginas na RAM a cada referência de memória.

Tabelas de Páginas Multinível

Divide a tabela em uma hierarquia de níveis, evitando alocar entradas para regiões do espaço de endereçamento que não são usadas.

Tabelas de Páginas Invertidas

Mantém uma única entrada por quadro físico (não por página virtual), exigindo busca associativa — normalmente acelerada por hashing.

Algoritmos de Substituição de Páginas (1/2)



Algoritmo	Lógica	Observação de Tanenbaum
Ótimo	Substitui a página que será referenciada mais tarde no futuro	Irrealizável — referência teórica de comparação
NRU	Classifica páginas em 4 classes pelos bits R/M; remove uma da classe mais baixa	Fácil de implementar, eficiência razoável
FIFO	Remove a página mais antiga na memória	Simples, mas remove páginas úteis só por serem antigas
Segunda Chance	Como FIFO, mas dá nova chance a páginas com bit R ligado	Corrige o defeito do FIFO puro
Relógio (Clock)	Implementação circular da segunda chance	Mais eficiente de implementar

Algoritmos de Substituição de Páginas (2/2)



Algoritmo	Lógica	Observação de Tanenbaum
LRU	Remove a página não usada há mais tempo (princípio da localidade)	Ótimo na prática, mas exige hardware especial e caro
NFU	Aproxima o LRU por software somando o bit R a um contador periodicamente	Não "esquece" o passado distante — distorce a decisão
Envelhecimento (Aging)	Desloca os contadores e soma R no bit mais significativo, simulando esquecimento	Boa aproximação de LRU com baixo custo
Working Set	Mantém na memória o conjunto de páginas usado ativamente no intervalo recente	Base teórica para combater o thrashing
WSClock	Combina working set com a estrutura circular do relógio	Um dos mais usados na prática (bom desempenho + viável)

Thrashing e o Conjunto de Trabalho



Thrashing (agitação)

O sistema gasta mais tempo trocando páginas do que executando processos — em geral por excesso de multiprogramação em relação à memória disponível.

Modelo do Conjunto de Trabalho

Conceito de Tanenbaum para prevenir o thrashing: o SO tenta manter na memória o conjunto de páginas que cada processo usa ativamente (o "working set"), controlando quantos processos podem permanecer ativos simultaneamente.

Questões de Projeto em Sistemas de Paginação



Alocação Local x Global

Restringir a vítima ao processo faltante ou escolher entre todas as páginas do sistema — políticas globais tendem a funcionar melhor.



Controle de Carga

Suspender temporariamente processos (swap out) quando há mais processos do que a memória suporta.



Tamanho de Página

Páginas pequenas reduzem fragmentação interna; páginas grandes reduzem overhead de tabela e faltas de página.



Páginas Compartilhadas

Bibliotecas e código reentrante podem ser mapeados por vários processos ao mesmo quadro físico.



Política de Limpeza

Um "paginador demoníaco" escreve previamente páginas modificadas, acelerando futuras faltas de página.



Interface de Memória Virtual

Em sistemas como o Mach, o processo participa do próprio gerenciamento, recebendo notificação de faltas.

Implementação: Tratando uma Falta de Página



1

Hardware causa interrupção e salva o contexto

2

Rotina em assembly salva os registradores gerais

3

SO identifica a página faltante e verifica sua legalidade

4

Escolhe-se um quadro vítima (grava-se em disco, se sujo)

5

A página requerida é lida do disco para o quadro

6

Tabela de páginas é atualizada e a instrução é reexecutada

Outros pontos de implementação: backup de instrução (o hardware informa onde a instrução começou, não só o byte que faltou) · travamento de páginas (pinning) para operações de E/S · armazenamento de apoio (backing store) em disco · separação entre política e mecanismo (algoritmo de substituição isolado do núcleo).

Segmentação



Em vez de um espaço de endereçamento linear dividido mecanicamente em páginas, o programa é dividido em múltiplos espaços independentes — os segmentos —, cada um representando uma entidade lógica (uma função, uma pilha, uma tabela).

- Simplifica estruturas de dados que crescem e encolhem de forma independente
- Facilita compilação separada e compartilhamento de código/dados
- Permite proteção naturalmente distinta por segmento (ex.: código só-execução)

Segmentação com Paginação

Cada segmento é, internamente, paginado — combinação usada historicamente pelo MULTICS e, de forma simplificada, por processadores x86 em modo legado. Une a organização lógica da segmentação à eficiência de alocação da paginação.

Paginação x Segmentação



Critério	Paginação	Segmentação
Visibilidade ao programador	Invisível — puramente mecânica	Visível — reflete a estrutura lógica do programa
Fragmentação	Interna (dentro da página)	Externa (exige compactação periódica)
Compartilhamento e proteção	Possível, mas menos natural	Natural por unidade lógica (segmento)
Uso combinado	Base da segmentação paginada (MULTICS, x86 legado)	Idem



Síntese

O capítulo caminha da ausência de abstração (um único processo, endereços físicos) até a memória virtual paginada com múltiplos níveis de tabela e algoritmos sofisticados de substituição, sempre equilibrando três forças:

Máquina Estendida

Ilusão de memória abundante e privada para cada processo

Gerenciador de Recursos

Uso eficiente de um recurso físico escasso e compartilhado

Restrições de Hardware

MMU, TLB e bits R/M determinam quais políticas são viáveis

Referências Bibliográficas



1

TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Capítulo 3 – Gerenciamento de Memória. ISBN 978-85-430-0567-6.

2

TANENBAUM, Andrew S.; BOS, Herbert. Modern Operating Systems. 4th ed. Boston: Pearson, 2014. Chapter 3 – Memory Management (edição original em inglês, usada para conferência de termos técnicos).

3

Wikipedia. Modern Operating Systems — histórico de edições da obra. Disponível em: en.wikipedia.org/wiki/Modern_Operating_Systems.

Material de apoio para o projeto "Sistemas Operacionais" — resumos, questionários e slides.